```
{
 "nbformat": 4,
 "nbformat_minor": 0,
 "metadata": {
  "colab": {
   "provenance": []
  },
  "kernelspec": {
   "name": "python3",
   "display_name": "Python 3"
  },
  "language_info": {
   "name": "python"
  }
 },
 "cells": [
  {
   "cell_type": "markdown",
   "metadata": {
    "id": "aTb-9TFFqprC"
   },
   "source": [
    "Importing the Dependencies"
   ]
  },
  {
   "cell_type": "code",
   "metadata": {
    "id": "3q9U3S_whh3-"
   },
   "source": [
    "import numpy as np\n",
    "import pandas as pd\n",
```

```
    "from sklearn.model_selection import train_test_split\n",
    "from sklearn.linear_model import LogisticRegression\n",
    "from sklearn.metrics import accuracy_score"
   ],
   "execution_count": null,
   "outputs": []
  },
  {
   "cell_type": "markdown",
   "metadata": {
    "id": "egMd5zeurTMR"
   },
   "source": [
    "Data Collection and Processing"
   ]
  },
  {
   "cell_type": "code",
   "metadata": {
    "id": "0q-3-LkQrREV"
   },
   "source": [
    "# loading the csv data to a Pandas DataFrame\n",
    "heart_data = pd.read_csv('/content/data.csv')"
   ],
   "execution_count": null,
   "outputs": []
  },
  {
   "cell_type": "code",
   "metadata": {
    "colab": {
     "base_uri": "https://localhost:8080/",
```

```
   "height": 198
  },
  "id": "M8dQxSTqriWD",
  "outputId": "ea695a74-7589-47fe-e400-2dd925f7b5bb"
 },
 "source": [
  "# print first 5 rows of the dataset\n",
  "heart_data.head()"
 ],
 "execution_count": null,
 "outputs": [
  {
   "output_type": "execute_result",
   "data": {
    "text/html": [
     "<div>\n",
     "<style scoped>\n",
     "    .dataframe tbody tr th:only-of-type {\n",
     "        vertical-align: middle;\n",
     "    }\n",
     "\n",
     "    .dataframe tbody tr th {\n",
     "        vertical-align: top;\n",
     "    }\n",
     "\n",
     "    .dataframe thead th {\n",
     "        text-align: right;\n",
     "    }\n",
     "</style>\n",
     "<table border=\"1\" class=\"dataframe\">\n",
     "  <thead>\n",
     "    <tr style=\"text-align: right;\">\n",
     "      <th></th>\n",
```

```
"    <th>age</th>\n",
"    <th>sex</th>\n",
"    <th>cp</th>\n",
"    <th>trestbps</th>\n",
"    <th>chol</th>\n",
"    <th>fbs</th>\n",
"    <th>restecg</th>\n",
"    <th>thalach</th>\n",
"    <th>exang</th>\n",
"    <th>oldpeak</th>\n",
"    <th>slope</th>\n",
"    <th>ca</th>\n",
"    <th>thal</th>\n",
"    <th>target</th>\n",
"   </tr>\n",
"  </thead>\n",
"  <tbody>\n",
"   <tr>\n",
"    <th>0</th>\n",
"    <td>63</td>\n",
"    <td>1</td>\n",
"    <td>3</td>\n",
"    <td>145</td>\n",
"    <td>233</td>\n",
"    <td>1</td>\n",
"    <td>0</td>\n",
"    <td>150</td>\n",
"    <td>0</td>\n",
"    <td>2.3</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>1</td>\n",
"    <td>1</td>\n",
```

```
"   </tr>\n",
"   <tr>\n",
"    <th>1</th>\n",
"    <td>37</td>\n",
"    <td>1</td>\n",
"    <td>2</td>\n",
"    <td>130</td>\n",
"    <td>250</td>\n",
"    <td>0</td>\n",
"    <td>1</td>\n",
"    <td>187</td>\n",
"    <td>0</td>\n",
"    <td>3.5</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>2</td>\n",
"    <td>1</td>\n",
"   </tr>\n",
"   <tr>\n",
"    <th>2</th>\n",
"    <td>41</td>\n",
"    <td>0</td>\n",
"    <td>1</td>\n",
"    <td>130</td>\n",
"    <td>204</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>172</td>\n",
"    <td>0</td>\n",
"    <td>1.4</td>\n",
"    <td>2</td>\n",
"    <td>0</td>\n",
"    <td>2</td>\n",
```

```
"    <td>1</td>\n",
"   </tr>\n",
"   <tr>\n",
"    <th>3</th>\n",
"    <td>56</td>\n",
"    <td>1</td>\n",
"    <td>1</td>\n",
"    <td>120</td>\n",
"    <td>236</td>\n",
"    <td>0</td>\n",
"    <td>1</td>\n",
"    <td>178</td>\n",
"    <td>0</td>\n",
"    <td>0.8</td>\n",
"    <td>2</td>\n",
"    <td>0</td>\n",
"    <td>2</td>\n",
"    <td>1</td>\n",
"   </tr>\n",
"   <tr>\n",
"    <th>4</th>\n",
"    <td>57</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>120</td>\n",
"    <td>354</td>\n",
"    <td>0</td>\n",
"    <td>1</td>\n",
"    <td>163</td>\n",
"    <td>1</td>\n",
"    <td>0.6</td>\n",
"    <td>2</td>\n",
"    <td>0</td>\n",
```

```
        "    <td>2</td>\n",
        "    <td>1</td>\n",
        "   </tr>\n",
        "  </tbody>\n",
        "</table>\n",
        "</div>"
       ],
       "text/plain": [
        "   age sex cp trestbps chol fbs ... exang oldpeak slope ca thal target\n
",
        "0  63   1  3     145  233   1 ...    0     2.3     0  0    1     1\n",
        "1  37   1  2     130  250   0 ...    0     3.5     0  0    2     1\n",
        "2  41   0  1     130  204   0 ...    0     1.4     2  0    2     1\n",
        "3  56   1  1     120  236   0 ...    0     0.8     2  0    2     1\n",
        "4  57   0  0     120  354   0 ...    1     0.6     2  0    2     1\n",
        "\n",
        "[5 rows x 14 columns]"
       ]
      },
      "metadata": {
       "tags": []
      },
      "execution_count": 3
     }
    ]
   },
   {
    "cell_type": "code",
    "metadata": {
     "colab": {
      "base_uri": "https://localhost:8080/",
      "height": 198
     },
```

```
  "id": "Fx_aCZDgrqdR",
  "outputId": "770eb646-bdff-45da-ac1c-06aae06f7446"
},
"source": [
  "# print last 5 rows of the dataset\n",
  "heart_data.tail()"
],
"execution_count": null,
"outputs": [
  {
    "output_type": "execute_result",
    "data": {
      "text/html": [
        "<div>\n",
        "<style scoped>\n",
        "    .dataframe tbody tr th:only-of-type {\n",
        "        vertical-align: middle;\n",
        "    }\n",
        "\n",
        "    .dataframe tbody tr th {\n",
        "        vertical-align: top;\n",
        "    }\n",
        "\n",
        "    .dataframe thead th {\n",
        "        text-align: right;\n",
        "    }\n",
        "</style>\n",
        "<table border=\"1\" class=\"dataframe\">\n",
        "  <thead>\n",
        "    <tr style=\"text-align: right;\">\n",
        "      <th></th>\n",
        "      <th>age</th>\n",
        "      <th>sex</th>\n",
```

```
"    <th>cp</th>\n",
"    <th>trestbps</th>\n",
"    <th>chol</th>\n",
"    <th>fbs</th>\n",
"    <th>restecg</th>\n",
"    <th>thalach</th>\n",
"    <th>exang</th>\n",
"    <th>oldpeak</th>\n",
"    <th>slope</th>\n",
"    <th>ca</th>\n",
"    <th>thal</th>\n",
"    <th>target</th>\n",
"   </tr>\n",
"  </thead>\n",
"  <tbody>\n",
"   <tr>\n",
"    <th>298</th>\n",
"    <td>57</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>140</td>\n",
"    <td>241</td>\n",
"    <td>0</td>\n",
"    <td>1</td>\n",
"    <td>123</td>\n",
"    <td>1</td>\n",
"    <td>0.2</td>\n",
"    <td>1</td>\n",
"    <td>0</td>\n",
"    <td>3</td>\n",
"    <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
```

```
"    <th>299</th>\n",
"    <td>45</td>\n",
"    <td>1</td>\n",
"    <td>3</td>\n",
"    <td>110</td>\n",
"    <td>264</td>\n",
"    <td>0</td>\n",
"    <td>1</td>\n",
"    <td>132</td>\n",
"    <td>0</td>\n",
"    <td>1.2</td>\n",
"    <td>1</td>\n",
"    <td>0</td>\n",
"    <td>3</td>\n",
"    <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"    <th>300</th>\n",
"    <td>68</td>\n",
"    <td>1</td>\n",
"    <td>0</td>\n",
"    <td>144</td>\n",
"    <td>193</td>\n",
"    <td>1</td>\n",
"    <td>1</td>\n",
"    <td>141</td>\n",
"    <td>0</td>\n",
"    <td>3.4</td>\n",
"    <td>1</td>\n",
"    <td>2</td>\n",
"    <td>3</td>\n",
"    <td>0</td>\n",
"   </tr>\n",
```

```
"   <tr>\n",
"    <th>301</th>\n",
"    <td>57</td>\n",
"    <td>1</td>\n",
"    <td>0</td>\n",
"    <td>130</td>\n",
"    <td>131</td>\n",
"    <td>0</td>\n",
"    <td>1</td>\n",
"    <td>115</td>\n",
"    <td>1</td>\n",
"    <td>1.2</td>\n",
"    <td>1</td>\n",
"    <td>1</td>\n",
"    <td>3</td>\n",
"    <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"    <th>302</th>\n",
"    <td>57</td>\n",
"    <td>0</td>\n",
"    <td>1</td>\n",
"    <td>130</td>\n",
"    <td>236</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>174</td>\n",
"    <td>0</td>\n",
"    <td>0.0</td>\n",
"    <td>1</td>\n",
"    <td>1</td>\n",
"    <td>2</td>\n",
"    <td>0</td>\n",
```

```
        "   </tr>\n",
        "  </tbody>\n",
        "</table>\n",
        "</div>"
       ],
       "text/plain": [
        "    age sex cp trestbps chol fbs ... exang oldpeak slope ca thal target\
n",
        "298  57   0  0     140  241   0 ...    1    0.2    1  0    3     0\n",
        "299  45   1  3     110  264   0 ...    0    1.2    1  0    3     0\n",
        "300  68   1  0     144  193   1 ...    0    3.4    1  2    3     0\n",
        "301  57   1  0     130  131   0 ...    1    1.2    1  1    3     0\n",
        "302  57   0  1     130  236   0 ...    0    0.0    1  1    2     0\n",
        "\n",
        "[5 rows x 14 columns]"
       ]
      },
      "metadata": {
       "tags": []
      },
      "execution_count": 4
     }
    ]
   },
   {
    "cell_type": "code",
    "metadata": {
     "colab": {
      "base_uri": "https://localhost:8080/"
     },
     "id": "8nX1tlzbrz0u",
     "outputId": "6f650e4c-22b6-4750-ca57-ba6758d1c3a2"
    },
```

```
 "source": [
  "# number of rows and columns in the dataset\n",
  "heart_data.shape"
 ],
 "execution_count": null,
 "outputs": [
  {
   "output_type": "execute_result",
   "data": {
    "text/plain": [
     "(303, 14)"
    ]
   },
   "metadata": {
    "tags": []
   },
   "execution_count": 5
  }
 ]
},
{
 "cell_type": "code",
 "metadata": {
  "colab": {
   "base_uri": "https://localhost:8080/"
  },
  "id": "7_xTcw1Sr6aJ",
  "outputId": "1948e00e-0656-43ef-c4c4-740ee51a6381"
 },
 "source": [
  "# getting some info about the data\n",
  "heart_data.info()"
 ],
```

```
 "execution_count": null,
 "outputs": [
  {
   "output_type": "stream",
   "text": [
    "<class 'pandas.core.frame.DataFrame'>\n",
    "RangeIndex: 303 entries, 0 to 302\n",
    "Data columns (total 14 columns):\n",
    " #   Column   Non-Null Count  Dtype  \n",
    "---  ------   --------------  -----  \n",
    " 0  age      303 non-null    int64  \n",
    " 1  sex      303 non-null    int64  \n",
    " 2  cp       303 non-null    int64  \n",
    " 3  trestbps  303 non-null    int64  \n",
    " 4  chol     303 non-null    int64  \n",
    " 5  fbs      303 non-null    int64  \n",
    " 6  restecg  303 non-null    int64  \n",
    " 7  thalach  303 non-null    int64  \n",
    " 8  exang    303 non-null    int64  \n",
    " 9  oldpeak  303 non-null    float64\n",
    " 10 slope    303 non-null    int64  \n",
    " 11 ca       303 non-null    int64  \n",
    " 12 thal     303 non-null    int64  \n",
    " 13 target   303 non-null    int64  \n",
    "dtypes: float64(1), int64(13)\n",
    "memory usage: 33.3 KB\n"
   ],
   "name": "stdout"
  }
 ]
},
{
 "cell_type": "code",
```

```
"metadata": {
 "colab": {
  "base_uri": "https://localhost:8080/"
 },
 "id": "GjHtW31rsGlb",
 "outputId": "8c1c23ce-b5b4-4872-a579-9b4185d12522"
},
"source": [
 "# checking for missing values\n",
 "heart_data.isnull().sum()"
],
"execution_count": null,
"outputs": [
 {
  "output_type": "execute_result",
  "data": {
   "text/plain": [
    "age       0\n",
    "sex       0\n",
    "cp        0\n",
    "trestbps  0\n",
    "chol      0\n",
    "fbs       0\n",
    "restecg   0\n",
    "thalach   0\n",
    "exang     0\n",
    "oldpeak   0\n",
    "slope     0\n",
    "ca        0\n",
    "thal      0\n",
    "target    0\n",
    "dtype: int64"
   ]
```

```
    },
    "metadata": {
     "tags": []
    },
    "execution_count": 7
   }
  ]
 },
 {
  "cell_type": "code",
  "metadata": {
   "colab": {
    "base_uri": "https://localhost:8080/",
    "height": 308
   },
   "id": "OHmcP7DJsSEP",
   "outputId": "400a121e-dbd2-4e77-8c72-021c12af5927"
  },
  "source": [
   "# statistical measures about the data\n",
   "heart_data.describe()"
  ],
  "execution_count": null,
  "outputs": [
   {
    "output_type": "execute_result",
    "data": {
     "text/html": [
      "<div>\n",
      "<style scoped>\n",
      "    .dataframe tbody tr th:only-of-type {\n",
      "        vertical-align: middle;\n",
      "    }\n",
```

```
"\n",
"    .dataframe tbody tr th {\n",
"        vertical-align: top;\n",
"    }\n",
"\n",
"    .dataframe thead th {\n",
"        text-align: right;\n",
"    }\n",
"</style>\n",
"<table border=\"1\" class=\"dataframe\">\n",
"  <thead>\n",
"    <tr style=\"text-align: right;\">\n",
"      <th></th>\n",
"      <th>age</th>\n",
"      <th>sex</th>\n",
"      <th>cp</th>\n",
"      <th>trestbps</th>\n",
"      <th>chol</th>\n",
"      <th>fbs</th>\n",
"      <th>restecg</th>\n",
"      <th>thalach</th>\n",
"      <th>exang</th>\n",
"      <th>oldpeak</th>\n",
"      <th>slope</th>\n",
"      <th>ca</th>\n",
"      <th>thal</th>\n",
"      <th>target</th>\n",
"    </tr>\n",
"  </thead>\n",
"  <tbody>\n",
"    <tr>\n",
"      <th>count</th>\n",
"      <td>303.000000</td>\n",
```

```
"    <td>303.000000</td>\n",
"    <td>303.000000</td>\n",
"    <td>303.000000</td>\n",
"    <td>303.000000</td>\n",
"    <td>303.000000</td>\n",
"    <td>303.000000</td>\n",
"    <td>303.000000</td>\n",
"    <td>303.000000</td>\n",
"    <td>303.000000</td>\n",
"    <td>303.000000</td>\n",
"    <td>303.000000</td>\n",
"    <td>303.000000</td>\n",
"    <td>303.000000</td>\n",
"    <td>303.000000</td>\n",
"   </tr>\n",
"   <tr>\n",
"    <th>mean</th>\n",
"    <td>54.366337</td>\n",
"    <td>0.683168</td>\n",
"    <td>0.966997</td>\n",
"    <td>131.623762</td>\n",
"    <td>246.264026</td>\n",
"    <td>0.148515</td>\n",
"    <td>0.528053</td>\n",
"    <td>149.646865</td>\n",
"    <td>0.326733</td>\n",
"    <td>1.039604</td>\n",
"    <td>1.399340</td>\n",
"    <td>0.729373</td>\n",
"    <td>2.313531</td>\n",
"    <td>0.544554</td>\n",
"   </tr>\n",
"   <tr>\n",
"    <th>std</th>\n",
```

```
"    <td>9.082101</td>\n",
"    <td>0.466011</td>\n",
"    <td>1.032052</td>\n",
"    <td>17.538143</td>\n",
"    <td>51.830751</td>\n",
"    <td>0.356198</td>\n",
"    <td>0.525860</td>\n",
"    <td>22.905161</td>\n",
"    <td>0.469794</td>\n",
"    <td>1.161075</td>\n",
"    <td>0.616226</td>\n",
"    <td>1.022606</td>\n",
"    <td>0.612277</td>\n",
"    <td>0.498835</td>\n",
"   </tr>\n",
"   <tr>\n",
"    <th>min</th>\n",
"    <td>29.000000</td>\n",
"    <td>0.000000</td>\n",
"    <td>0.000000</td>\n",
"    <td>94.000000</td>\n",
"    <td>126.000000</td>\n",
"    <td>0.000000</td>\n",
"    <td>0.000000</td>\n",
"    <td>71.000000</td>\n",
"    <td>0.000000</td>\n",
"    <td>0.000000</td>\n",
"    <td>0.000000</td>\n",
"    <td>0.000000</td>\n",
"    <td>0.000000</td>\n",
"    <td>0.000000</td>\n",
"   </tr>\n",
"   <tr>\n",
```

```
"    <th>25%</th>\n",
"    <td>47.500000</td>\n",
"    <td>0.000000</td>\n",
"    <td>0.000000</td>\n",
"    <td>120.000000</td>\n",
"    <td>211.000000</td>\n",
"    <td>0.000000</td>\n",
"    <td>0.000000</td>\n",
"    <td>133.500000</td>\n",
"    <td>0.000000</td>\n",
"    <td>0.000000</td>\n",
"    <td>1.000000</td>\n",
"    <td>0.000000</td>\n",
"    <td>2.000000</td>\n",
"    <td>0.000000</td>\n",
"   </tr>\n",
"   <tr>\n",
"    <th>50%</th>\n",
"    <td>55.000000</td>\n",
"    <td>1.000000</td>\n",
"    <td>1.000000</td>\n",
"    <td>130.000000</td>\n",
"    <td>240.000000</td>\n",
"    <td>0.000000</td>\n",
"    <td>1.000000</td>\n",
"    <td>153.000000</td>\n",
"    <td>0.000000</td>\n",
"    <td>0.800000</td>\n",
"    <td>1.000000</td>\n",
"    <td>0.000000</td>\n",
"    <td>2.000000</td>\n",
"    <td>1.000000</td>\n",
"   </tr>\n",
```

```
"   <tr>\n",
"    <th>75%</th>\n",
"    <td>61.000000</td>\n",
"    <td>1.000000</td>\n",
"    <td>2.000000</td>\n",
"    <td>140.000000</td>\n",
"    <td>274.500000</td>\n",
"    <td>0.000000</td>\n",
"    <td>1.000000</td>\n",
"    <td>166.000000</td>\n",
"    <td>1.000000</td>\n",
"    <td>1.600000</td>\n",
"    <td>2.000000</td>\n",
"    <td>1.000000</td>\n",
"    <td>3.000000</td>\n",
"    <td>1.000000</td>\n",
"   </tr>\n",
"   <tr>\n",
"    <th>max</th>\n",
"    <td>77.000000</td>\n",
"    <td>1.000000</td>\n",
"    <td>3.000000</td>\n",
"    <td>200.000000</td>\n",
"    <td>564.000000</td>\n",
"    <td>1.000000</td>\n",
"    <td>2.000000</td>\n",
"    <td>202.000000</td>\n",
"    <td>1.000000</td>\n",
"    <td>6.200000</td>\n",
"    <td>2.000000</td>\n",
"    <td>4.000000</td>\n",
"    <td>3.000000</td>\n",
"    <td>1.000000</td>\n",
```

```
        "   </tr>\n",
        "  </tbody>\n",
        "</table>\n",
        "</div>"
       ],
       "text/plain": [
        "             age      sex       cp ...       ca      thal    target\n",
        "count 303.000000 303.000000 303.000000 ... 303.000000 303.000000 303.000000\n",
        "mean   54.366337   0.683168   0.966997 ...   0.729373   2.313531   0.544554\n",
        "std     9.082101   0.466011   1.032052 ...   1.022606   0.612277   0.498835\n",
        "min    29.000000   0.000000   0.000000 ...   0.000000   0.000000   0.000000\n",
        "25%    47.500000   0.000000   0.000000 ...   0.000000   2.000000   0.000000\n",
        "50%    55.000000   1.000000   1.000000 ...   0.000000   2.000000   1.000000\n",
        "75%    61.000000   1.000000   2.000000 ...   1.000000   3.000000   1.000000\n",
        "max    77.000000   1.000000   3.000000 ...   4.000000   3.000000   1.000000\n",
        "\n",
        "[8 rows x 14 columns]"
       ]
      },
      "metadata": {
       "tags": []
      },
      "execution_count": 8
     }
    ]
```

```
},
{
 "cell_type": "code",
 "metadata": {
  "colab": {
   "base_uri": "https://localhost:8080/"
  },
  "id": "4lnaOSlUsfWP",
  "outputId": "6c38694f-7445-47b3-e235-cdd0157a4ec6"
 },
 "source": [
  "# checking the distribution of Target Variable\n",
  "heart_data['target'].value_counts()"
 ],
 "execution_count": null,
 "outputs": [
  {
   "output_type": "execute_result",
   "data": {
    "text/plain": [
     "1   165\n",
     "0   138\n",
     "Name: target, dtype: int64"
    ]
   },
   "metadata": {
    "tags": []
   },
   "execution_count": 10
  }
 ]
},
{
```

```
 "cell_type": "markdown",
 "metadata": {
  "id": "aSOBu4qDtJy5"
 },
 "source": [
  "1 --> Defective Heart\n",
  "\n",
  "0 --> Healthy Heart"
 ]
},
{
 "cell_type": "markdown",
 "metadata": {
  "id": "tW8i4igjtPRC"
 },
 "source": [
  "Splitting the Features and Target"
 ]
},
{
 "cell_type": "code",
 "metadata": {
  "id": "Q6yfbswrs7m3"
 },
 "source": [
  "X = heart_data.drop(columns='target', axis=1)\n",
  "Y = heart_data['target']"
 ],
 "execution_count": null,
 "outputs": []
},
{
 "cell_type": "code",
```

```
"metadata": {
 "colab": {
  "base_uri": "https://localhost:8080/"
 },
 "id": "XJoCp4ZKtpZy",
 "outputId": "549bc077-393d-4763-f64f-3d2e5faa0c7f"
},
"source": [
 "print(X)"
],
"execution_count": null,
"outputs": [
 {
  "output_type": "stream",
  "text": [
   "     age  sex  cp  trestbps  chol  ...  exang  oldpeak  slope  ca  thal\n",
   "0    63    1   3     145   233  ...      0      2.3      0   0     1\n",
   "1    37    1   2     130   250  ...      0      3.5      0   0     2\n",
   "2    41    0   1     130   204  ...      0      1.4      2   0     2\n",
   "3    56    1   1     120   236  ...      0      0.8      2   0     2\n",
   "4    57    0   0     120   354  ...      1      0.6      2   0     2\n",
   "..   ...  ...  ..     ...   ...  ...    ...      ...    ...  ..   ...\n",
   "298  57    0   0     140   241  ...      1      0.2      1   0     3\n",
   "299  45    1   3     110   264  ...      0      1.2      1   0     3\n",
   "300  68    1   0     144   193  ...      0      3.4      1   2     3\n",
   "301  57    1   0     130   131  ...      1      1.2      1   1     3\n",
   "302  57    0   1     130   236  ...      0      0.0      1   1     2\n",
   "\n",
   "[303 rows x 13 columns]\n"
  ],
  "name": "stdout"
 }
]
```

```
},
{
 "cell_type": "code",
 "metadata": {
  "colab": {
   "base_uri": "https://localhost:8080/"
  },
  "id": "nukuj-Yltq1w",
  "outputId": "7c604a47-1690-4db4-fec7-bed3e9497428"
 },
 "source": [
  "print(Y)"
 ],
 "execution_count": null,
 "outputs": [
  {
   "output_type": "stream",
   "text": [
    "0    1\n",
    "1    1\n",
    "2    1\n",
    "3    1\n",
    "4    1\n",
    "     ..\n",
    "298  0\n",
    "299  0\n",
    "300  0\n",
    "301  0\n",
    "302  0\n",
    "Name: target, Length: 303, dtype: int64\n"
   ],
   "name": "stdout"
  }
```

```
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {
    "id": "_EcjSE3Et18n"
   },
   "source": [
    "Splitting the Data into Training data & Test Data"
   ]
  },
  {
   "cell_type": "code",
   "metadata": {
    "id": "a-UUfRUxtuga"
   },
   "source": [
    "X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, stratify=
Y, random_state=2)"
   ],
   "execution_count": null,
   "outputs": []
  },
  {
   "cell_type": "code",
   "metadata": {
    "colab": {
     "base_uri": "https://localhost:8080/"
    },
    "id": "x7PrjC6zuf6X",
    "outputId": "f2d66421-d671-4475-a51c-b37de3a2edac"
   },
   "source": [
```

```
   "print(X.shape, X_train.shape, X_test.shape)"
  ],
  "execution_count": null,
  "outputs": [
   {
    "output_type": "stream",
    "text": [
     "(303, 13) (242, 13) (61, 13)\n"
    ],
    "name": "stdout"
   }
  ]
 },
 {
  "cell_type": "markdown",
  "metadata": {
   "id": "beSkZmpVuvn9"
  },
  "source": [
   "Model Training"
  ]
 },
 {
  "cell_type": "markdown",
  "metadata": {
   "id": "gi2NOWZjuxzw"
  },
  "source": [
   "Logistic Regression"
  ]
 },
 {
  "cell_type": "code",
```

```
    "metadata": {
     "id": "4-Md74FYuqNL"
    },
    "source": [
     "model = LogisticRegression()"
    ],
    "execution_count": null,
    "outputs": []
   },
   {
    "cell_type": "code",
    "metadata": {
     "colab": {
      "base_uri": "https://localhost:8080/"
     },
     "id": "kCdHYxGUu7XD",
     "outputId": "ff7185b7-1dd2-418d-c22f-778005b4655b"
    },
    "source": [
     "# training the LogisticRegression model with Training data\n",
     "model.fit(X_train, Y_train)"
    ],
    "execution_count": null,
    "outputs": [
     {
      "output_type": "stream",
      "text": [
       "/usr/local/lib/python3.7/dist-packages/sklearn/linear_model/_logistic.py:9
40: ConvergenceWarning: lbfgs failed to converge (status=1):\n",
       "STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.\n",
       "\n",
       "Increase the number of iterations (max_iter) or scale the data as shown in:\
n",
```

```
        "    https://scikit-learn.org/stable/modules/preprocessing.html\n",
        "Please also refer to the documentation for alternative solver options:\n",
        "    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression\n",
        "  extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)\n"
       ],
       "name": "stderr"
      },
      {
       "output_type": "execute_result",
       "data": {
        "text/plain": [
         "LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,\n",
         "                   intercept_scaling=1, l1_ratio=None, max_iter=100,\n",
         "                   multi_class='auto', n_jobs=None, penalty='l2',\n",
         "                   random_state=None, solver='lbfgs', tol=0.0001, verbose=0,\n",
         "                   warm_start=False)"
        ]
       },
       "metadata": {
        "tags": []
       },
       "execution_count": 17
      }
     ]
    },
    {
     "cell_type": "markdown",
     "metadata": {
      "id": "ZYlw8Gi9vXfU"
     },
     "source": [
```

```
   "Model Evaluation"
  ]
 },
 {
  "cell_type": "markdown",
  "metadata": {
   "id": "wmxAekfZvZa9"
  },
  "source": [
   "Accuracy Score"
  ]
 },
 {
  "cell_type": "code",
  "metadata": {
   "id": "g19JaUTMvPKy"
  },
  "source": [
   "# accuracy on training data\n",
   "X_train_prediction = model.predict(X_train)\n",
   "training_data_accuracy = accuracy_score(X_train_prediction, Y_train)"
  ],
  "execution_count": null,
  "outputs": []
 },
 {
  "cell_type": "code",
  "metadata": {
   "colab": {
    "base_uri": "https://localhost:8080/"
   },
   "id": "uQBZvBh8v7R_",
   "outputId": "e798e765-9f84-43e2-d3a0-f0fea3192ac2"
```

```
 },
 "source": [
  "print('Accuracy on Training data : ', training_data_accuracy)"
 ],
 "execution_count": null,
 "outputs": [
  {
   "output_type": "stream",
   "text": [
    "Accuracy on Training data :  0.8512396694214877\n"
   ],
   "name": "stdout"
  }
 ]
},
{
 "cell_type": "code",
 "metadata": {
  "id": "mDONDJdlwBlO"
 },
 "source": [
  "# accuracy on test data\n",
  "X_test_prediction = model.predict(X_test)\n",
  "test_data_accuracy = accuracy_score(X_test_prediction, Y_test)"
 ],
 "execution_count": null,
 "outputs": []
},
{
 "cell_type": "code",
 "metadata": {
  "colab": {
   "base_uri": "https://localhost:8080/"
```

```
   },
   "id": "_MBS-OqdwYpf",
   "outputId": "a2f5bd57-7135-47c2-c8f2-01f1823ad66a"
  },
  "source": [
   "print('Accuracy on Test data : ', test_data_accuracy)"
  ],
  "execution_count": null,
  "outputs": [
   {
    "output_type": "stream",
    "text": [
     "Accuracy on Test data :  0.8196721311475410\n"
    ],
    "name": "stdout"
   }
  ]
 },
 {
  "cell_type": "markdown",
  "metadata": {
   "id": "jlruVh3Qwq0e"
  },
  "source": [
   "Building a Predictive System"
  ]
 },
 {
  "cell_type": "code",
  "metadata": {
   "colab": {
    "base_uri": "https://localhost:8080/"
   },
```

```
    "id": "9ercruC9wb4C",
    "outputId": "6a7f8964-d7c5-4a54-bb76-ef18f2add04a"
   },
   "source": [
    "input_data = (62,0,0,140,268,0,0,160,0,3.6,0,2,2)\n",
    "\n",
    "# change the input data to a numpy array\n",
    "input_data_as_numpy_array= np.asarray(input_data)\n",
    "\n",
    "# reshape the numpy array as we are predicting for only on instance\n",
    "input_data_reshaped = input_data_as_numpy_array.reshape(1,-1)\n",
    "\n",
    "prediction = model.predict(input_data_reshaped)\n",
    "print(prediction)\n",
    "\n",
    "if (prediction[0]== 0):\n",
    "  print('The Person does not have a Heart Disease')\n",
    "else:\n",
    "  print('The Person has Heart Disease')"
   ],
   "execution_count": null,
   "outputs": [
    {
     "output_type": "stream",
     "text": [
      "[0]\n",
      "The Person does not have a Heart Disease\n"
     ],
     "name": "stdout"
    }
   ]
  }
]
```

```
}
```